

02 — Descripción de la Persistencia de Datos

Proyecto: Plataforma Libro de Clases Digital
Asignatura: DSY1106 — Desarrollo Fullstack III
Evaluación: Parcial N°3 — Encargo (base) · Actualizado para Examen 2026-07-13
Equipo: Cristian Monsalve / Héctor Olivares

Examen: las alertas a apoderados persisten como **mensajes** en `librodigital_academic` (ver sección 4.2 y documento 05).

1. Estrategia general

El proyecto adopta el patrón **Database per Service**: cada microservicio posee su propia base de datos PostgreSQL, gestionada de forma independiente. Esto garantiza:

- **Autonomía** de cada dominio (auth, académico, asistencia)
- **Escalabilidad** independiente por servicio
- **Aislamiento de fallos** entre dominios
- **Evolución de esquema** sin bloquear otros servicios

No existen **foreign keys entre bases distintas**. Las relaciones cruzadas (por ejemplo, `student_id` en attendance que referencia un estudiante en academic) se resuelven mediante **identificadores lógicos** validados en la capa de servicio o vía llamadas REST.

2. Inventario de bases de datos

Base de datos	Microservicio	Puerto	Script DDL
<code>librodigital_auth</code>	authService	8091	<code>infraestructura/ddl/auth_schema.sql</code>
<code>librodigital_academic</code>	academicService	8092	<code>infraestructura/ddl/academic_schema.sql</code>
<code>librodigital_attendance</code>	attendanceService	8093	<code>infraestructura/ddl/attendance_schema.sql</code>

Creación inicial de las tres bases:

```
-- infraestructura/ddl/00_create_databases.sql
CREATE DATABASE librodigital_auth;
CREATE DATABASE librodigital_academic;
CREATE DATABASE librodigital_attendance;
```

3. Tecnología de persistencia: Spring Data JPA + Hibernate

3.1 Configuración por microservicio

Cada servicio define su conexión en `application.properties` :

```
# Ejemplo authService
spring.datasource.url=jdbc:postgresql://localhost:5432/librodigital_auth
spring.datasource.username=postgres
spring.datasource.password=****
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
```

Propiedad	Propósito
<code>spring.datasource.url</code>	URL JDBC a la BD del servicio
<code>spring.jpa.hibernate.ddl-auto=update</code>	Sincroniza esquema con entidades JPA en desarrollo
<code>PostgreSQLDialect</code>	Generación SQL compatible con PostgreSQL

En producción se recomienda `ddl-auto=validate` y aplicar cambios solo vía scripts versionados en `infraestructura/ddl/`.

3.2 Capas de acceso a datos

```
Controller (REST)
  ↓
Service / ServiceImpl ← lógica de negocio y validaciones
  ↓
Repository (JpaRepository) ← acceso a datos
  ↓
Entidad JPA (@Entity) ← mapeo objeto-relacional
```

↓
PostgreSQL

Ejemplo — authService:

```
@Entity
@Table(name = "users")
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String password; // BCrypt hash
    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles", ...)
    private Set<Role> roles;
}

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
}
```

3.3 Patrón Repository

Spring Data JPA genera implementaciones automáticas de repositorios. Ventajas:

- Consultas derivadas del nombre del método (`findByFirstName`)
- Integración con transacciones (`@Transactional`)
- Facilita pruebas unitarias con mocks (`@Mock UserRepository`)

4. Esquemas por servicio

4.1 librodigital_auth (authService)

Diagrama Entidad-Relación - AuthService (librodigital_auth)

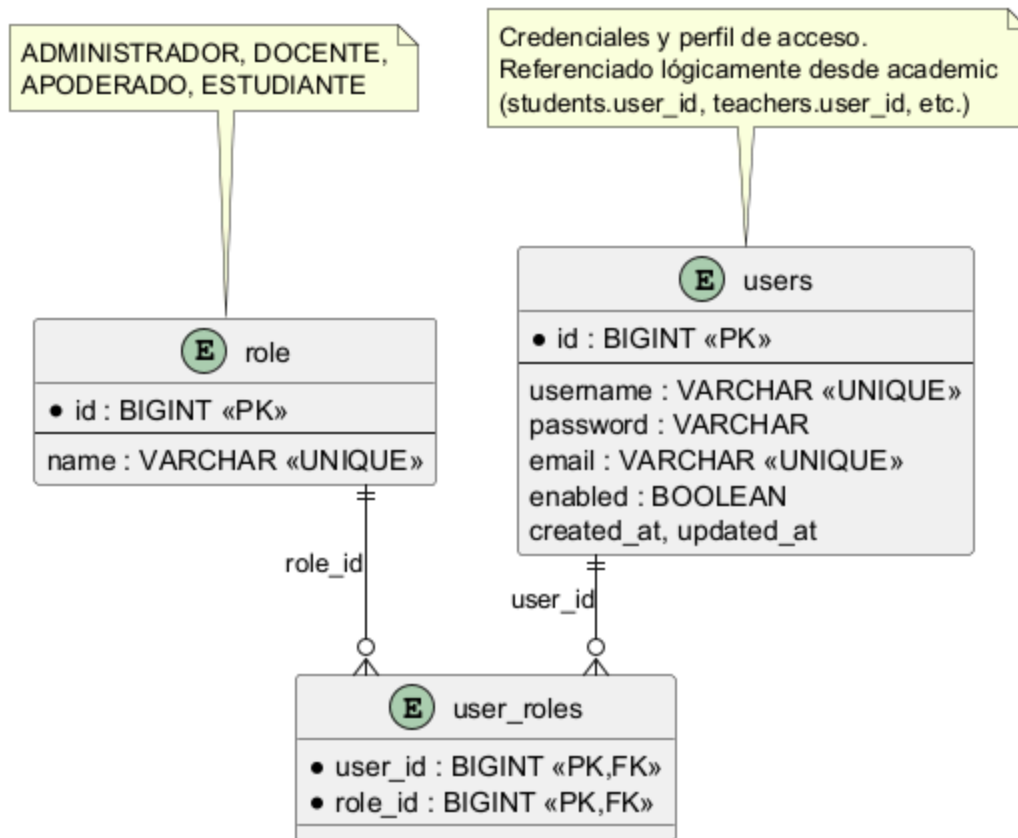
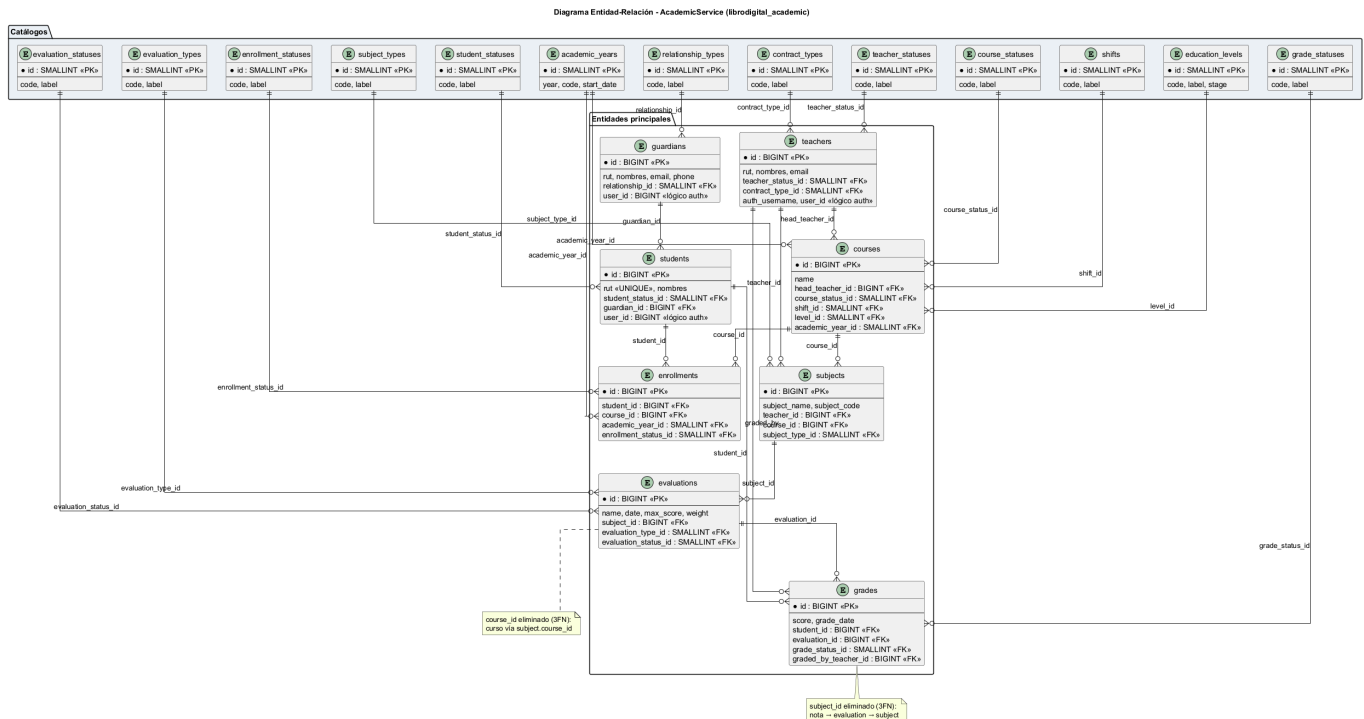


Tabla	Descripción
users	Credenciales, email, estado activo
role	Catálogo de roles (ADMIN , DOCENTE , APODERADO , ESTUDIANTE)
user_roles	Relación N:M usuario ↔ rol

Normalización: 3FN — los roles están en tabla separada; no hay redundancia de nombres de rol en users .

4.2 librodigital_academic (academicService)



El esquema académico fue **normalizado a Tercera Forma Normal (3FN)**:

- **Catálogos** reemplazan columnas VARCHAR categóricas:
 - student_statuses, teacher_statuses, course_statuses, enrollment_statuses
 - evaluation_types, evaluation_statuses, grade_statuses
 - relationship_types, contract_types, subject_types, shifts
 - education_levels, academic_years
- **Entidades principales** con claves foráneas internas:
 - guardians, teachers, students
 - courses, subjects, enrollments
 - evaluations, grades
 - conversations, messages, conversation_read_states (mensajería interna + alertas a apoderados)

Beneficio de la normalización: elimina redundancia, garantiza integridad referencial dentro del dominio académico y facilita consultas por catálogo.

Mensajería (post-EP3): los avisos automáticos de asistencia y de nuevas evaluaciones se guardan como mensajes en conversaciones tipo TG (docente–apoderado–alumno). No requieren una cuarta base de datos: el dominio de comunicación familiar vive en academic.

4.3 librodigital_attendance (attendanceService)

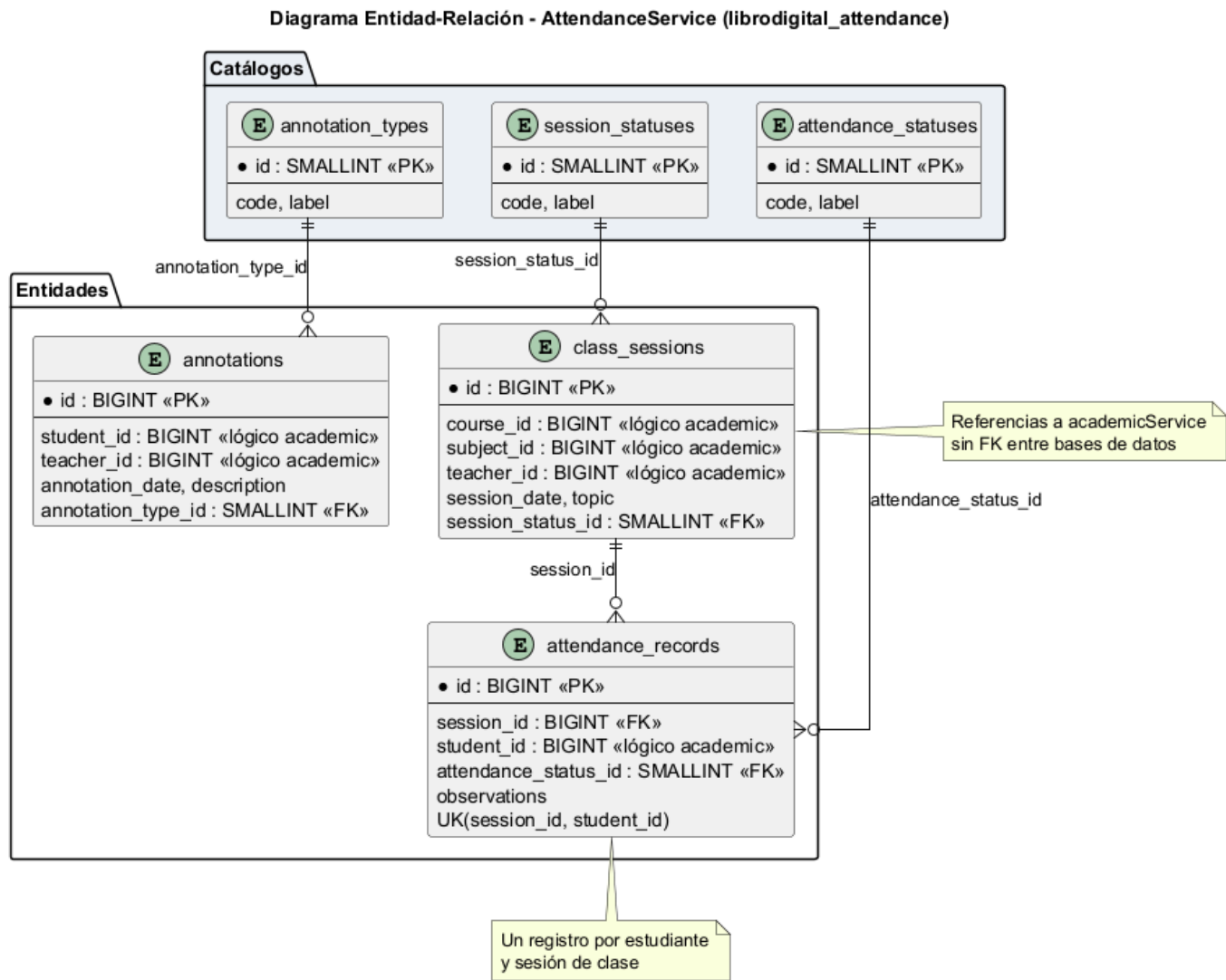


Tabla	Descripción
class_sessions	Sesión de clase (fecha, curso, asignatura, docente — IDs lógicos; puede incluir sustituto)
attendance_records	Registro de asistencia por estudiante y sesión (P/A/R/J)
annotations	Anotaciones de conducta vinculadas a estudiante/sesión

Al **guardar** un `attendance_record`, el servicio puede invocar `academicService` para notificar al apoderado; **no** duplica el mensaje en la BD de attendance.

Referencias lógicas cross-service:

Campo en attendance	Referencia lógica	Servicio origen
course_id	Curso	academicService
subject_id	Asignatura	academicService
teacher_id	Docente	academicService
student_id	Estudiante	academicService

Estos IDs se validan en la capa de servicio; no hay FK física hacia `librodigital_academic`.

5. Scripts SQL y migraciones

Ubicación: `infraestructura/ddl/`

Archivo	Uso
<code>00_create_databases.sql</code>	Crear las 3 bases
<code>auth_schema.sql</code>	Esquema completo auth + datos demo
<code>academic_schema.sql</code>	Esquema académico 3FN + catálogos + demo
<code>attendance_schema.sql</code>	Esquema asistencia + catálogos + demo
<code>migrations/001-004</code>	Solo para actualizar BDs creadas con esquema antiguo

Instalación nueva: ejecutar `00_create_databases.sql` y luego cada `*_schema.sql`.

6. Procedimientos almacenados

En esta etapa del proyecto **no se utilizan procedimientos almacenados (SP)**. La persistencia se implementa íntegramente con:

- **JPA/Hibernate** para CRUD y consultas derivadas
- **JPQL / @Query** cuando se requieren consultas personalizadas
- **Scripts DDL** versionados para instalación y migraciones

Esta decisión prioriza la portabilidad del código Java y la coherencia con el patrón Repository de Spring.

7. Transacciones y consistencia

- Cada operación de escritura en un servicio se ejecuta dentro de una **transacción local** (`@Transactional` en la capa Service).
- No hay transacciones distribuidas (2PC) entre microservicios.
- La **consistencia eventual** entre dominios se logra mediante:
 - Validación de IDs antes de persistir (attendance verifica que `student_id` exista vía API o reglas de negocio)
 - Diseño de APIs idempotentes donde corresponde

8. Seguridad de datos

Aspecto	Implementación
Contraseñas	Hash BCrypt en <code>authService</code> ; nunca en texto plano
Tokens	JWT firmado; sin estado en servidor (stateless)
Exposición API	DTOs separados de entidades; no se serializan campos sensibles
Conexión BD	Credenciales en <code>application.properties</code> (variables de entorno en producción)

9. Datos de demostración

Usuarios de prueba (auth):

Usuario	Rol	Contraseña
<code>admin_colegio</code>	SUPER_ADMINISTRADOR (Dirección)	<code>test1234</code>
<code>admin_oficina</code>	ADMINISTRATIVO (Oficina)	<code>test1234</code>
<code>prof_castillo</code>	DOCENTE	<code>test1234</code>
<code>apoderado_demo</code>	APODERADO	<code>test1234</code>
<code>estudiante_demo</code>	ESTUDIANTE	<code>test1234</code>

Los scripts `*_schema.sql` incluyen registros demo para academic y attendance. Tras crear evaluaciones o pasar lista, los avisos aparecen como filas en `messages` (academic).

10. Conclusión

La persistencia del sistema se implementa con **JPA/Hibernate sobre PostgreSQL**, siguiendo **Database per Service** y **normalización 3FN** en los dominios académico y de asistencia. Los scripts DDL en `infraestructura/ddl/` documentan el esquema físico; las entidades JPA en cada microservicio mantienen la sincronización en desarrollo. Esta arquitectura garantiza **separación de responsabilidades**, **integridad referencial intra-servicio** y **escalabilidad** acorde a los requisitos del caso Libro Digital.